Oracle SOA Integration for C2M (Web Services & APIs)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — SOA Integration for C2M (Web Services & APIs) 4

The problem . . . 4

What this solution does . . . 4

How it's delivered . . . 4

The business outcome . . . 4

Who it's for . . . 5

Solution Overview — SOA Integration for C2M (Web Services & APIs) 5

The C2M integration surface — three mechanisms . . . 5

Naming convention used throughout . . . 7

How the mechanisms work together . . . 7

Consistent, supportable, observable — the point of the solution . . . 7

Deployment model . . . 8

Version note . . . 8

Sources . . . 8

Requirements — SOA Integration for C2M 9

1. A running C2M/OUAF WebLogic domain with IWS deployable . . . 9

2. Oracle SOA middleware tier available (SOA Suite / Service Bus) — where SOA is used . . . 9

3. Integration inventory & direction defined (what talks to what) . . . 10

4. Inbound Web Service records defined and enabled (for inbound flows) . . . 10

5. Outbound Message configuration defined (for outbound flows) . . . 10

6. Endpoints, credentials & connectivity for each integration . . . 11

7. Consistent authN, error-handling & observability standard agreed . . . 11

8. Backups and a rollback point before changes . . . 12

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 12

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 12

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 13

3. Environment topology (Dev / Test / Prod) & access . . . 13

4. Administrative access & tooling (WLST, shell, SQL) . . . 13

5. Change control, backup & a non-production pilot . . . 13

6. Collaborating teams & support model . . . 14

Implementation Guide — Designing & Deploying C2M SOA Integration 14

Phase 0 — Assess (read-only) . . . 15

Phase 1 — Establish the integration standards (design once, reuse everywhere) . . . 15

Phase 2 — Deploy & enable Inbound Web Services (inbound surface) . . . 16

Phase 3 — Configure Outbound Messages (outbound surface) . . . 16

Phase 4 — Build & deploy the SOA composites / adapters (middleware tier) . . . 17
Phase 5 — Validate the integration end to end (Pilot) . . . 18
Phase 6 — Expand to remaining environments . . . 18
Rollback . . . 19
Sources . . . 19
Testing — Validating C2M SOA Integration 20
A. Inbound Web Services (IWS) . . . 20
B. Outbound Messages . . . 20
C. SOA composites / middleware . . . 21
D. End-to-end integration . . . 22
E. Consolidated health . . . 22
Troubleshooting — C2M SOA Integration 23
Inbound Web Services (IWS) . . . 23
Outbound Messages . . . 24
SOA composites / middleware . . . 25
Cross-cutting . . . 25
Escalation . . . 26
Sources . . . 26
Best Practices — C2M SOA Integration (supportable, low-lock-in) 26
Consistency (design once, reuse everywhere) . . . 26
Supportability . . . 27
Observability . . . 27
Security (consistent authN) . . . 28
Error handling & resilience . . . 28
Low lock-in . . . 28
Cross-cutting . . . 28
Sources . . . 29
Deliverables Summary — SOA Integration for C2M 29
Documentation . . . 29
Automation scripts (`scripts/`) . . . 30
How they work together . . . 31
Outcome . . . 31
Appendix A — PowerShell Scripts 32

Executive Summary — SOA Integration for C2M (Web Services & APIs)

The problem

Oracle Utilities **Customer to Meter (C2M)** rarely stands alone — it exchanges data with meters, billing, analytics, CRM, and enterprise systems, and much of that runs through **Oracle SOA** and web services/APIs. When each integration is built as a one-off — different naming, no common authentication, inconsistent error handling, no way to see whether a flow actually worked — the result is a fragile, unsupportable snowflake estate. Failures are business-visible (an inbound call fails, an outbound event never reaches the external system) and hard to diagnose because nothing is consistent or observable.

What this solution does

It defines and deploys the **C2M SOA/web-service integration surface as one coherent, consistent unit**, pilot-first:

- **Inbound Web Services (IWS)** — external systems call C2M as **SOAP/REST** through the deployed `spl-web-services` application, with only the required services enabled and secured. (IWS is the supported replacement for the retired XAI layer.)
- **Outbound Messages** — C2M emits events through the right transport per interface: **real-time** HTTP/SOAP senders, the **SOA** processing method (polled by Oracle Service Bus via the OUAF Outbound Message transport), or **JMS** — each reconciled so messages actually dispatch.
- **SOA composites & adapters** — named, reusable composites/pipelines that mediate, transform, and route between C2M and external systems using standard adapters, with fault handling and instance tracking.

All of it built to **one** standard for **authentication, error-handling, and observability** — so every integration is **consistent, supportable, and observable**. Delivered as **self-guided documentation plus five automation scripts** (WLST/Jython, shell, SQL — the authentic C2M/WebLogic/SOA toolchain) for discovery, validation, reporting, safe remediation, and scoping.

How it's delivered

The proven **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model: all changes are made on a **non-production environment first**, behind explicit pass/fail gates, with a documented rollback, before promoting the **same named objects and same scripts** to Test and Production. Patterns favour open SOAP/REST and standard adapters to keep **lock-in low** — the mediation tier could be replaced without re-engineering C2M.

The business outcome

C2M integrated with the enterprise through **consistent, supportable, observable patterns** — IWS, Outbound Messages, and SOA composites — proven on a pilot before you scale, with saved evidence for change control. Self-guided and tested, at a fraction of the cost of a bespoke consulting engagement.

Who it's for

C2M/OUAF administrators, WebLogic middleware engineers, and integration/SOA teams standing up or rationalising the C2M integration surface on WebLogic 14c (reference stack C2M 25.4 / minimum v2.9). Oracle SOA Suite / Service Bus deep internals are deferred to Oracle's SOA documentation; version-specific details are cited or flagged "verify for your version", and My-Oracle-Support-gated notes are pointed to, not reproduced.

Solution Overview — SOA Integration for C2M (Web Services & APIs)

Oracle Utilities **Customer to Meter (C2M)** rarely stands alone. Even though C2M is a single-instance product that folds **CC&B**, **MDM**, and service-order/device management into one **Oracle Utilities Application Framework (OUAF)** stack, in the enterprise it still exchanges data with meters, billing, analytics, CRM, and other systems. A great deal of that exchange runs through **Oracle SOA** and web services/APIs. This solution defines a **consistent, supportable, observable SOA integration approach for C2M**: the composites and adapters between C2M and other systems, the inbound/outbound web services (**IWS**), and the API patterns — so integrations stop being one-off snowflakes and become a repeatable platform capability.

> ***What this solution is and is not.*** *This is an* ***integration architecture and administration*** *solution: how to design, name, deploy, secure, and observe the SOA/web-service integration surface of C2M using Oracle's supported mechanisms. It is* ***not*** *an Oracle SOA Suite internals course — deep BPEL/Mediator/OSB engine internals are deferred to Oracle's SOA Suite / Service Bus documentation and are cited, not reproduced.*

The C2M integration surface — three mechanisms

C2M exposes and consumes integration through three mechanisms that this solution treats as one coherent surface:

Mechanism	Direction	OUAF / SOA object	What it enables
Inbound Web Services (IWS)	**Into** C2M	OUAF **Inbound Web Service** metadata + the `spl-web-services` EAR on WebLogic	External systems call C2M business services as **SOAP or REST** endpoints. IWS is the supported replacement for the retired **XAI** inbound layer.

Mechanism	Direction	OUAF / SOA object	What it enables
Outbound Messages	**Out of** C2M	OUAF **Outbound Message Type** + **External System** + **Message Sender**; for SOA the processing method is **SOA**	C2M emits events/requests to external systems — real-time (HTTP/web-service sender) or asynchronous (JMS, or the **OUAF Outbound Message** OSB transport).
SOA composites / adapters	Both	Oracle **SOA Suite / Service Bus** composites, **JCA adapters** (HTTP/SOAP, JMS, DB, File) and the **Oracle Utilities / SOA Suite adapters**	The middleware tier that mediates, transforms, routes, and orchestrates between C2M and other systems.

Inbound: IWS (SOAP & REST)

IWS is the OUAF mechanism that exposes C2M business services (business services, service scripts, BOs) as web-service operations. Since C2M **no longer supports XAI**, all inbound integration into C2M is via IWS. Inbound REST web services support **GET, PATCH, POST, PUT, DELETE**; SOAP inbound services are described by a WSDL. The container is the `spl-web-services` Java EE application deployed to WebLogic with the OUAF `iwsdeploy` utility. External callers — including SOA composites — invoke these endpoints.

Outbound: Outbound Messages (real-time & asynchronous)

C2M initiates integration by creating an **Outbound Message** (typically from an algorithm, BO, or service script). Routing is governed by the **External System** and the **Outbound Message Type**, whose **processing method** selects the transport:

• **Real-time** — a **Message Sender** of a real-time class (an HTTP/SOAP sender that calls an external web service synchronously).

• **SOA/OSB** — the processing method **SOA**: the outbound message lands in the **outbound message staging table**, and Oracle Service Bus polls it asynchronously via the product's** OUAF Outbound Message** custom transport (a proxy service per outbound message type routes to a business service).

• **JMS** — a JMS message sender hands the payload to a queue/topic for asynchronous consumption.

The middleware tier: SOA composites & adapters

Between C2M and the wider estate sits **Oracle SOA Suite / Oracle Service Bus**. Integrations are built as **composites/services** that use **JCA adapters** (SOAP/HTTP, JMS, Database, File) and, where present, the **Oracle Utilities Adapter** (a SOAP/REST catalog of OUAF inbound/outbound services) or the **SOA Suite Adapter**. The middleware is where cross-system concerns live: transformation, routing, retry, and correlation — so C2M stays clean and the integration logic is externalised and reusable.

Naming convention used throughout

So the docs, scripts, and console/WLST steps line up, this solution refers to a single **named set of objects** and reuses those names everywhere. **Substitute your own environment's standard names — but pick them once and reuse them, exactly as shown here.**

Role	Reference name
C2M External System (SOA target)	`C2M-SOA-EXT`
Outbound Message Type (SOA)	`C2M-OMT-SOA` (processing method **SOA**)
Real-time HTTP/SOAP Message Sender	`C2M-RTHTTP-SNDR`
JMS Message Sender	`C2M-JMS-SNDR` (connection factory `jms/C2MConnectionFactory`, queue `jms/C2MOutboundQueue`)
Representative Inbound Web Service	`C2M-IWS-CustomerSync`
IWS container application (EAR)	`spl-web-services`
SOA composite (inbound → C2M)	`C2MInboundComposite`
SOA composite (C2M → external)	`C2MOutboundComposite`
Oracle Service Bus proxy (per OMT)	`C2M-OMT-SOA-Proxy` → business service `C2M-OMT-SOA-BizSvc`

How the mechanisms work together

A typical **inbound** flow: an external system calls `C2MInboundComposite` → the composite transforms and invokes the C2M IWS `C2M-IWS-CustomerSync` on `spl-web-services` → C2M processes it. A typical **outbound** flow: C2M raises an **Outbound Message** on type `C2M-OMT-SOA` → OSB's `C2M-OMT-SOA-Proxy` polls the staging table via the **OUAF Outbound Message** transport → transforms → routes through `C2M-OMT-SOA-BizSvc` to the external system. Real-time flows instead use the `C2M-RTHTTP-SNDR` sender to call an external web service synchronously; asynchronous flows use `C2M-JMS-SNDR` onto a JMS queue.

Consistent, supportable, observable — the point of the solution

- **Consistent** — one naming standard, one authN model, one error-handling pattern, one observability approach applied to every integration, rather than per-interface improvisation.
- **Supportable** — named, inventoried composites/services/IWS that an operator can discover, validate, and reason about; the automation scripts inventory and check exactly these.
- **Observable** — every flow emits enough (SOA/OSB instance tracking, C2M outbound-message status, IWS request logging) to answer "did it work, and if not, where did it fail?".

• **Low lock-in** — favour standard SOAP/REST over WebLogic and open JCA adapters, keep transformation in

the middleware tier (not buried in C2M), and name/document interfaces so the middleware could be swapped (e.g. to Oracle Integration / OIC) without re-engineering C2M.

Deployment model

Like every Oracle solution in this catalogue, work proceeds **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, on a **non-production environment first**, with explicit pass/fail gates and a documented rollback before any production change. The included automation is **WLST (Jython), shell, and SQL** — never PowerShell — because that is the authentic C2M/WebLogic/SOA toolchain.

Version note

• **C2M 25.4** (minimum **v2.9**) on **Oracle WebLogic Server 14c** is the reference stack; Oracle SOA

Suite / Oracle Service Bus versions are those certified alongside it. Earlier releases differ (notably the **XAI** → **IWS** transition — XAI is no longer supported). Where a menu path, utility flag, adapter capability, or default differs by release, the step is flagged **"verify for your version"**.

• Some integration notes, adapter downloads, and patches are entitlement-gated on **My Oracle Support

(MOS) **— for example** Doc ID 1644914.1 — Migrating from XAI to IWS**; this solution points to them rather than reproducing gated content.

Sources

• Oracle Utilities — *Polling Outbound Messages Using OSB* (OUAF Outbound Message transport, processing

method SOA, proxy/business service, outbound message staging table): https://docs.oracle.com/en/industries/utilities/meter-solution-cloud-service/264/mscs-user-guides/Topics/F1_93XAI_Polling_Outbound_Message_Using_OSB.html

• Oracle Utilities C2M — *Inbound REST Web Services* (IWS overview; HTTP methods; XAI retired):

https://docs.oracle.com/en/industries/energy-water/advanced-meter/2.9.0.0.0/c2m-user-guides/C2M_29000/F1_93XAI_REST_Inbound_Web_Service_Overview.html

• Oracle Utilities C2M — *Deploying Inbound Web Services (IWS)* (installation guide; `iwsdeploy`,

`spl-web-services`): https://docs.oracle.com/en/industries/energy-water/advanced-meter/29011/c2m-installation-guide/C2M-INSTALLATION-GUIDE/C2M_Install_Postinstall.09.09.html

• Oracle® Fusion Middleware — *Understanding Technology Adapters* / *Oracle JCA Adapter for JMS* (SOA

Suite JCA adapters): https://docs.oracle.com/en/middleware/soa-suite/integration-adapters/12.2.1.3/develop-soa-adapters/overview3.html

• Oracle — *Using the Oracle Utilities Adapter with Oracle Integration* (SOAP/REST catalog of OUAF

inbound/outbound services; trigger consumes outbound message, invoke calls inbound web services): https://docs.oracle.com/en/cloud/paas/application-integration/utilities-adapters/oracle-utilities-adapter-capabilities.html

- My Oracle Support — *Migrating from XAI to IWS*, Doc ID 1644914.1 (entitlement-gated).

Requirements — SOA Integration for C2M

These are the **solution-specific** prerequisites for defining and deploying the C2M SOA/web-service integration surface. They are in addition to the **Global Requirements** for every Oracle solution (licensed C2M + WebLogic entitlement, certified platform stack, Dev/Test/Prod topology, WLST/shell/SQL tooling, change control + backup + non-prod pilot, and collaborating teams) — verify those first.

Each requirement states **Description**, **Why it's required**, **How to verify**, and **How to implement if missing**.

1. A running C2M/OUAF WebLogic domain with IWS deployable

- **Description:** An installed, startable OUAF/C2M WebLogic domain with the Administration Server and at

least one managed server hosting the C2M application, and the environment configured so the `spl-web-services` (IWS) EAR can be deployed with the OUAF `iwsdeploy` utility.

- **Why required:** IWS is the inbound integration surface; SOA composites call C2M through it. There is

nothing to integrate into if the domain and IWS container are not in place.

- **How to verify:** Connect with WLST (`connect()`); confirm managed server(s) reach **RUNNING** and

the C2M application is targeted. Confirm web services are enabled in `configureEnv` and `$SPLEBASE/bin/iwsdeploy.sh` exists.

- **How to implement if missing:** Complete the *WebLogic 14c C2M Install & Configure* and enable IWS

(`configureEnv` → `initialSetup`) before starting here. XAI is retired — use IWS.

2. Oracle SOA middleware tier available (SOA Suite / Service Bus) — where SOA is used

- **Description:** Where the integration design uses SOA/OSB, a reachable **Oracle SOA Suite / Oracle

Service Bus** domain (Enterprise Manager / OSB console accessible) certified alongside the C2M release, with deployment access to it.

- **Why required:** Composites, proxy/business services, and the **OUAF Outbound Message** OSB transport

live in the middleware tier; you cannot deploy or observe them without it. (Purely IWS/direct-HTTP integrations may not need SOA — scope this per interface.)

- **How to verify:** Log in to the SOA/OSB Enterprise Manager / Service Bus console; confirm you can list

and deploy composites/projects; confirm the **OUAF Outbound Message** transport is installed on OSB if the SOA processing method will be used.

- **How to implement if missing:** Stand up / obtain access to the SOA Suite / Service Bus environment

and install the OUAF Outbound Message transport, or design the interface to use IWS/real-time HTTP instead (a valid lower-lock-in choice for many flows).

3. Integration inventory & direction defined (what talks to what)

- **Description:** A written inventory of the integrations in scope — for each, the **direction**

(inbound to C2M / outbound from C2M / both), the **external system**, the **payload**, and the required **timing** (real-time synchronous vs. asynchronous/near-real-time).

- **Why required:** The mechanism (IWS vs. outbound message vs. SOA composite vs. JMS) follows from

direction and timing. Building endpoints without this inventory produces the snowflake interfaces this solution exists to prevent.

- **How to verify:** Confirm an integration register exists listing each interface, its direction, its

external system, and its timing requirement.

- **How to implement if missing:** Work with the integration and business teams to produce the register

before designing endpoints; the `OSOAC2M-ScopeHelper` script consumes this to fix the target set.

4. Inbound Web Service records defined and enabled (for inbound flows)

- **Description:** The specific OUAF **Inbound Web Service** metadata records for the in-scope inbound

integrations (SOAP and/or REST), each **active/enabled**, backed by the intended business service/service script/BO, e.g. the reference `C2M-IWS-CustomerSync`.

- **Why required:** Deploying `spl-web-services` makes the container available, but each inbound service

must be defined and enabled to respond; enabling everything needlessly expands the attack/support surface.

- **How to verify:** In C2M, list the **Inbound Web Service** records and their status; retrieve the

WSDL/endpoint for each in-scope service and confirm it loads.

- **How to implement if missing:** Define/enable only the required Inbound Web Service records and

redeploy `spl-web-services` so the EAR reflects them. Migrate any legacy XAI inbound service to IWS (MOS **Doc ID 1644914.1**).

5. Outbound Message configuration defined (for outbound flows)

- **Description:** For each outbound integration, the OUAF **External System** (reference

`C2M-SOA-EXT`), **Outbound Message Type** (reference `C2M-OMT-SOA`), and the **Message Sender** with the correct **processing method** — **SOA** for OSB-polled asynchronous flows, a **real-time HTTP/SOAP**

sender (`C2M-RTHTTP-SNDR`) for synchronous calls, or a **JMS** sender (`C2M-JMS-SNDR`) for asynchronous queue hand-off.

• **Why required:** C2M routes outbound messages by external system + outbound message type; the processing method selects the transport. A mismatch means messages stage but never dispatch, or dispatch to the wrong place.

• **How to verify:** In C2M, review the **External System**, **Outbound Message Type**, and **Message Sender** definitions; confirm the processing method matches the intended transport and that the sender points at the right endpoint/queue.

• **How to implement if missing:** Define the External System, Outbound Message Type (processing method SOA/real-time/JMS as designed), and Message Sender before wiring the middleware.

6. Endpoints, credentials & connectivity for each integration

• **Description:** For every interface, the **endpoint URL** (C2M IWS endpoints for inbound; external system URLs/queues for outbound), the **credentials** (service accounts / keystores), and confirmed **network reachability** (firewall/proxy) between C2M, the SOA/OSB tier, and the external systems.

• **Why required:** Integrations fail most often on connectivity and credentials, not logic. These must be known and reachable before deployment, and secrets must be handled securely (never in source).

• **How to verify:** From the SOA/OSB host, confirm the C2M IWS endpoint is reachable and authenticates; from C2M, confirm the outbound endpoint/queue is reachable; confirm certificates are trusted (OUAF trust store / `initialSetup -i`).

• **How to implement if missing:** Open the connectivity, provision the service accounts and keystores, and store credentials in the environment/secret store — not in scripts or committed files.

7. Consistent authN, error-handling & observability standard agreed

• **Description:** A single agreed standard applied to **every** integration: the **authentication** model (HTTP Basic over TLS, WS-Security/WS-Policy, OAuth where supported — *verify for your version*), the **error-handling** pattern (retry/redelivery limits, error queues, C2M outbound-message error status handling, fault policies in SOA), and the **observability** approach (SOA/OSB instance tracking, C2M outbound-message status, IWS request logging, and the monitoring stack the environment uses).

• **Why required:** Consistency is the whole point — one standard makes every integration supportable and observable instead of each one behaving differently. Ad-hoc per-interface choices are unsupportable.

• **How to verify:** Confirm the standard is documented and that a reference integration already conforms to it (authN enforced, errors captured, flow observable end to end).

• **How to implement if missing:** Agree and document the standard (authN + error-handling +

observability) **before** building interfaces, then apply it to every one; the Best Practices guide gives the recommended defaults.

8. Backups and a rollback point before changes

• **Description:** A captured baseline before changes — domain `config.xml`, the `$SPLEBASE` product/config (IWS, external system, outbound message, message sender metadata), and the SOA/OSB project/composite exports (SAR/JAR/configjar) — plus a change window for production.

• **Why required:** IWS deployment, C2M integration metadata, and SOA/OSB deployments are all production-impacting; a clean restore path is the rollback.

• **How to verify:** Confirm current backups of the domain config, product home, and SOA/OSB project exports, and that you can restore/redeploy them.

• **How to implement if missing:** Take the backups (export the SOA/OSB projects) and secure the change approval before proceeding (per the global change-control requirement).

Gate to begin implementation: all eight items above pass on the **pilot (non-production)** environment, the object names are agreed (Overview naming convention), the authN/error/observability standard is documented, and a rollback is in hand.

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified

version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain

config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

Implementation Guide — Designing & Deploying C2M SOA Integration

This guide designs and deploys the C2M SOA/web-service integration surface end to end: the **named SOA composites/adapters**, the **inbound (IWS)** and **outbound (Outbound Message)** flows, a **consistent authN / error-handling / observability** standard, and validation of the integrations. It follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, on the **pilot (non-production)** environment first, with pass/fail gates and a rollback.

Object names below match the Overview naming convention and are reused throughout; substitute your environment's standard names but keep them consistent across C2M config, WLST, and the SOA/OSB console.

> ***Reference names:*** *External System* `C2M-SOA-EXT`*; Outbound Message Type* `C2M-OMT-SOA` *(processing method **SOA**); real-time sender* `C2M-RTHTTP-SNDR`*; JMS sender* `C2M-JMS-SNDR` *(*`jms/C2MConnectionFactory` */* `jms/C2MOutboundQueue`*); Inbound Web Service* `C2M-IWS-CustomerSync`*; IWS EAR* `spl-web-services`*; composites* `C2MInboundComposite` */* `C2MOutboundComposite`*; OSB proxy* `C2M-OMT-SOA-Proxy` *→ business service* `C2M-OMT-SOA-BizSvc`*.*

> ***Safety rules for every phase.*** *Take config + SOA/OSB backups first (Requirement 8). IWS (re)deployment and some C2M metadata changes require redeploy/regeneration; do disruptive work in a maintenance window; keep the rollback (restore* `config.xml` */* `$SPLEBASE`*, redeploy prior SOA*

*projects) ready. **Never** put credentials or keystores in scripts or committed files.*

Phase 0 — Assess (read-only)

Goal: capture the current integration surface before changing anything.

1. Connect WLST read-only and record: managed-server states, whether `spl-web-services` is deployed and

where, and any JMS resources used by integration.

1. Run `OSOAC2M-Discovery` (read-only) to snapshot the integration surface — deployed IWS/EAR,

integration-relevant JMS, and (from the C2M schema) the External Systems, Outbound Message Types, Message Senders, and Inbound Web Service records — to an evidence file. Run `OSOAC2M-ScopeHelper` to fix the exact target set (which interfaces/servers/SOA projects you will touch) so later scripts act only in scope.

1. Confirm all Requirements pass on the pilot environment, and that the authN/error/observability

standard (Requirement 7) is documented.

Gate 0 (pass/fail): you have a written baseline of the integration surface, an agreed target scope, the integration inventory (Requirement 3), and a rollback point. Do not proceed until all pass.

Phase 1 — Establish the integration standards (design once, reuse everywhere)

Goal: the consistent patterns that every subsequent interface will follow.

1. **Naming standard.** Adopt the Overview naming convention for External Systems, Outbound Message

Types, Message Senders, Inbound Web Services, composites, and OSB proxy/business services. Record it so discovery/validation scripts can assert it.

1. **Authentication standard.** Decide one model and apply it to every interface: **TLS everywhere**;

inbound IWS secured with HTTP Basic over TLS or **WS-Security/WS-Policy** (SOAP), OAuth where the release supports it (*verify for your version*); outbound senders authenticate to external systems with a dedicated service account/keystore. Import certificates into the OUAF trust store with `initialSetup -i`.

1. **Error-handling standard.** Define retry/redelivery limits, an **error destination** for JMS,

fault policies for SOA composites, and how C2M **outbound-message error status** is monitored and retried. No interface silently drops on failure.

1. **Observability standard.** Turn on **SOA/OSB instance tracking** (composite instances, OSB pipeline

reporting), rely on **C2M outbound-message status** for outbound flows, enable **IWS request logging** for inbound, and wire each into the environment's monitoring stack so every flow answers "did it work, and if not, where?".

Gate 1: the four standards are documented and a reference interface will be built to them.

Phase 2 — Deploy & enable Inbound Web Services (inbound surface)

Goal: the `spl-web-services` EAR deployed and the in-scope inbound services responding to callers (including SOA composites).

1. **Enable web services in the environment.** From `$SPLEBASE/bin`, run `configureEnv` (`-a`) and set

the IWS parameters — the **IWS Deployment Target** and the **Web Admin Server Host**. Leave the development-environment (browser-deploy) flag **off** in production. Process the changes (`P`).

1. **Run initial setup.** From `$SPLEBASE/bin` run `initialSetup` so the templated config

(`weblogic.xml`, `web.xml`, `spl.properties`, etc.) is regenerated for IWS.

1. **Prepare headless WebLogic credentials.** Ensure `$SPLEBASE/etc/.wlsuserconfig` and `.wlsuserkey`

exist (created with WLST `storeUserConfig`) so deploys run without an interactive password. Import the WebLogic certificate into the OUAF trust store with `initialSetup -i` if TLS is in play.

1. **Deploy the EAR.** With the Admin Server (and target managed server) **RUNNING**, from

`$SPLEBASE/bin`:

```
cd $SPLEBASE/bin
ksh ./iwsdeploy.sh        # deploys the spl-web-services EAR to the IWS Deployment Target
```

(On Windows the equivalent is `iwsdeploy.cmd`.) The Admin Server **must be up** before deploying.

1. **Define and enable the required Inbound Web Services.** In C2M, create/activate the **Inbound Web

Service **records for the in-scope inbound integrations (reference `C2M-IWS-CustomerSync`), backed by the intended business service/service script/BO — SOAP or REST as designed. Enable** only** what integrations need; redeploy if the service list changed.

1. **Apply the authN standard** to each IWS endpoint (TLS + Basic/WS-Security/OAuth per Phase 1).
2. **Verify** with `OSOAC2M-Validation`: `spl-web-services` is **deployed/active** on the target, and

each in-scope inbound service's WSDL/endpoint responds to a well-formed, authenticated test call.

Gate 2: `spl-web-services` active; each in-scope inbound service enabled, secured, and returning a well-formed response (or WSDL); disabled/out-of-scope services are not exposed.

Phase 3 — Configure Outbound Messages (outbound surface)

Goal: C2M emits outbound integration through the correct transport per interface.

1. **Create the External System.** In C2M, define the external system `C2M-SOA-EXT` representing the

integration target (or one external system per real target).

1. **Create the Outbound Message Type & set the processing method.** Define `C2M-OMT-SOA` and set the

processing method:

• **SOA** — for asynchronous/near-real-time flows polled by Oracle Service Bus. The outbound message lands in the **outbound message staging table**; OSB polls it via the **OUAF Outbound Message** custom transport (Phase 4).

• **Real-time** — attach the HTTP/SOAP **Message Sender** `C2M-RTHTTP-SNDR` for a synchronous call to an external web service.

• **JMS** — attach the JMS **Message Sender** `C2M-JMS-SNDR` (connection factory `jms/C2MConnectionFactory`, destination `jms/C2MOutboundQueue`) for asynchronous queue hand-off.

1. **Create the Message Sender(s)** referenced above, pointing at the correct endpoint URL / JNDI destination, with the agreed authN and the error/retry settings from the standard.

1. **Wire the trigger.** Confirm the C2M algorithm/BO/service script that raises the outbound message references `C2M-OMT-SOA` and `C2M-SOA-EXT` so events actually stage.

1. **Reconcile transport vs. design.** A real-time interface must use a real-time sender; an asynchronous one the SOA/OSB or JMS path. A mismatch stages messages that never dispatch.

1. **Verify** with `OSOAC2M-Validation` / `OSOAC2M-Report`: the External System, Outbound Message Type (processing method), and Message Sender exist and reconcile; for SOA flows, staged messages are visible for OSB to poll.

Gate 3: outbound configuration in place and reconciled — external system, outbound message type (correct processing method), and sender all consistent with the interface's direction and timing.

Phase 4 — Build & deploy the SOA composites / adapters (middleware tier)

Goal: named, reusable composites/services that mediate between C2M and external systems, using standard adapters and the agreed patterns.

1. **Inbound composite (`C2MInboundComposite`).** In SOA Suite / Service Bus, build a composite that receives from the external system (SOAP/HTTP or JCA adapter), transforms, and **invokes the C2M IWS** `C2M-IWS-CustomerSync` — using the **Oracle Utilities Adapter** (a SOAP/REST catalog of OUAF inbound/outbound services) where available, or a plain SOAP/HTTP reference otherwise. Keep transformation in the composite, not in C2M.

1. **Outbound composite / OSB pipeline (`C2MOutboundComposite`).** For the **SOA** processing method, build the **OUAF Outbound Message** flow on OSB: a **proxy service** `C2M-OMT-SOA-Proxy` (one per outbound message type) that polls the staging table via the OUAF Outbound Message transport, performs transformation, and routes through the **business service** `C2M-OMT-SOA-BizSvc` to the external system.

1. **Use standard JCA adapters** (SOAP/HTTP, JMS, Database, File) for external connectivity so the integration stays on open, portable mechanisms rather than bespoke code.

1. **Apply fault policies and instance tracking** (the error-handling + observability standards) to every

composite/pipeline: retry/fault handling on faults, instance tracking on so flows are traceable in Enterprise Manager / the OSB console.

1. **Deploy** the composite(s)/OSB project(s) to the middleware tier (keep the exported SAR/JAR/configjar

as the rollback artifact), configure endpoints/credentials from the environment/secret store — **not** in the project — and enable.

1. **Verify** with `OSOAC2M-Validation`: `spl-web-services` reachable from the middleware host, the

composites/proxies deployed and active, and a test instance flows end to end.

Gate 4: `C2MInboundComposite`, `C2MOutboundComposite` (and the OSB proxy/business service) deployed and active, using standard adapters, fault policies, and instance tracking; endpoints/credentials sourced securely.

Phase 5 — Validate the integration end to end (Pilot)

Goal: prove inbound + outbound + middleware together on the pilot before expanding.

1. Run `OSOAC2M-Validation` across all in-scope interfaces for one consolidated pass on IWS

deployment/health, outbound-message configuration reconciliation, and composite/proxy status.

1. Run `OSOAC2M-Report` (read-only) to produce a health/coverage report — which IWS are enabled and

responding, which External System/Outbound Message Type/Sender sets reconcile, which composites are active, and any staged-but-undispatched outbound messages.

1. Execute the functional checks in **05-testing.md** (an inbound call drives C2M; an outbound event

dispatches through the correct transport; the authN/error/observability standard holds).

1. If anything is off, correct with `OSOAC2M-Remediation` (**dry-run by default** — review the

planned actions, then re-run with the explicit apply flag inside the change window).

Gate 5: the report shows the inbound, outbound, and middleware surfaces healthy on the pilot and the functional tests pass.

Phase 6 — Expand to remaining environments

1. Repeat Phases 2–5 per environment (Test → Prod), each behind its own change window, backup, and

rollback. Promote the **same named objects** and redeploy the **same** SOA/OSB project exports.

1. Use `OSOAC2M-ScopeHelper` to retarget the scripts to each environment's interface/server set so you

run the **same** validated steps everywhere.

1. In production, keep the development-environment IWS flag **off**, enable only required inbound

services, source all credentials from the secret store, and confirm the report is clean before closing the change.

Final gate: every target environment shows a clean `OSOAC2M-Report` — IWS responding, outbound dispatching through the right transport, composites active — with a captured rollback point.

Rollback

- **IWS:** undeploy `spl-web-services` (or redeploy the prior EAR) and disable the newly enabled inbound

services.

- **Outbound Messages:** revert the External System / Outbound Message Type / Message Sender metadata

from the `$SPLEBASE` backup (or disable the newly added ones).

- **SOA/OSB:** redeploy the previous composite/OSB project export (SAR/JAR/configjar); undeploy the new

one.

- Re-run `OSOAC2M-Validation` to confirm the environment is back to its pre-change baseline.

Sources

- Oracle Utilities — *Polling Outbound Messages Using OSB* (processing method SOA, OUAF Outbound Message

transport, proxy/business service, outbound message staging table): https://docs.oracle.com/en/industries/utilities/meter-solution-cloud-service/264/mscs-user-guides/Topics/F1_93XAI_Polling_Outbound_Message_Using_OSB.html

- Oracle Utilities C2M — *Deploying Inbound Web Services (IWS)* (`iwsdeploy`, `spl-web-services`,

`configureEnv`/`initialSetup`): https://docs.oracle.com/en/industries/energy-water/advanced-meter/29011/c2m-installation-guide/C2M-INSTALLATION-GUIDE/C2M_Install_Postinstall.09.09.html

- Oracle Utilities C2M — *Inbound REST Web Services* (IWS SOAP/REST; HTTP methods; XAI retired):

https://docs.oracle.com/en/industries/energy-water/advanced-meter/2.9.0.0.0/c2m-user-guides/C2M_29000/F1_93XAI_REST_Inbound_Web_Service_Overview.html

- Oracle® Fusion Middleware — *Understanding Technology Adapters* / JCA adapters (SOAP/HTTP, JMS, DB,

File) for SOA composites: https://docs.oracle.com/en/middleware/soa-suite/integration-adapters/12.2.1.3/develop-soa-adapters/overview3.html

- Oracle — *Using the Oracle Utilities Adapter with Oracle Integration* (SOAP/REST catalog; trigger

consumes outbound message, invoke calls inbound web services): https://docs.oracle.com/en/cloud/paas/application-integration/utilities-adapters/oracle-utilities-adapter-capabilities.html

- My Oracle Support — *Migrating from XAI to IWS*, Doc ID 1644914.1 (entitlement-gated).

Testing — Validating C2M SOA Integration

These tests prove the integration surface works **individually and together** — inbound (IWS), outbound (Outbound Messages), and the SOA/OSB middleware — on the pilot environment first. Each test states the **objective**, **steps**, and an explicit **pass criterion**. Run the read-only scripts (`OSOAC2M-Discovery`, `OSOAC2M-Validation`, `OSOAC2M-Report`) for repeatable evidence; capture output as change-record proof.

> *Run tests on **non-production first**. A failing gate here is a stop — fix the cause (see 06-troubleshooting.md) before promoting.*

A. Inbound Web Services (IWS)

A1 — EAR deployed and active

- **Objective:** the web-services container is live on the IWS Deployment Target.
- **Steps:** run `OSOAC2M-Validation`; or console *Deployments*.
- **Pass:** `spl-web-services` is **Active/OK** on the intended target server.

A2 — Inbound service responds and is secured

- **Objective:** an enabled inbound service answers an authenticated call.
- **Steps:** retrieve the WSDL/endpoint for `C2M-IWS-CustomerSync`; issue a well-formed test request

(SOAP or REST) with **valid** credentials over TLS, then repeat with **invalid**/missing credentials.

- **Pass:** the valid call returns a well-formed, expected response (or the WSDL loads); the

unauthenticated/invalid call is **rejected** (authN standard enforced); disabled/out-of-scope services are not exposed.

A3 — REST method coverage (where REST is used)

- **Objective:** the inbound REST service honours only its defined operations.
- **Steps:** call the defined HTTP methods (from **GET/PATCH/POST/PUT/DELETE**) for the service and one

undefined method.

- **Pass:** defined operations behave as specified; an undefined method is refused — no unintended

surface.

B. Outbound Messages

B1 — Outbound configuration reconciles

• **Objective:** the external system, outbound message type, and sender are consistent.

• **Steps:** run `OSOAC2M-Report`; review `C2M-SOA-EXT`, `C2M-OMT-SOA` (processing method), and the referenced Message Sender.

• **Pass:** the External System, Outbound Message Type, and Message Sender all exist and reconcile; the processing method matches the interface's intended transport (SOA / real-time / JMS).

B2 — Real-time outbound dispatches synchronously

• **Objective:** a real-time interface calls the external web service and returns.

• **Steps:** trigger the C2M action that raises an outbound message on a **real-time** type using `C2M-RTHTTP-SNDR`; observe the outbound-message status and the external system's receipt.

• **Pass:** the message dispatches synchronously, the outbound-message status shows success, and the external system received it; a forced failure is captured as an **error status** (not a silent drop).

B3 — SOA/OSB outbound is polled and dispatched

• **Objective:** an asynchronous (processing method **SOA**) message is picked up by OSB.

• **Steps:** trigger an outbound message on `C2M-OMT-SOA`; confirm it lands in the **outbound message staging table**; confirm the OSB `C2M-OMT-SOA-Proxy` polls it (OUAF Outbound Message transport) and routes via `C2M-OMT-SOA-BizSvc` to the external system.

• **Pass:** the staged message is consumed by OSB in near-real-time and delivered; the OSB instance is visible in tracking; no message is left staged-but-undispatched.

B4 — JMS outbound hand-off (where JMS is used)

• **Objective:** an asynchronous JMS interface enqueues correctly.

• **Steps:** trigger an outbound message on the **JMS** sender `C2M-JMS-SNDR`; observe `jms/C2MOutboundQueue` pending/consumed counts.

• **Pass:** the message is enqueued and then consumed (pending returns to baseline); the JNDI destination matches what the sender references.

C. SOA composites / middleware

C1 — Composites deployed and active

• **Objective:** the named middleware artifacts are live.

• **Steps:** run `OSOAC2M-Validation`; or the SOA Enterprise Manager / OSB console.

• **Pass:** `C2MInboundComposite`, `C2MOutboundComposite`, and the OSB proxy/business service are

deployed and **active**.

C2 — Instance tracking & fault handling work

• **Objective:** flows are observable and faults are handled per the standard.

• **Steps:** run a normal instance and a deliberately faulting instance (e.g. bad payload / unreachable target); inspect instance tracking and the configured **fault policy** / error destination.

• **Pass:** the normal instance is traceable end to end in tracking; the faulting instance is caught by the fault policy / lands in the error destination and is retried per the standard — **not** lost.

D. End-to-end integration

D1 — Inbound path drives real C2M behaviour

• **Objective:** an external call through the middleware changes C2M.

• **Steps:** call `C2MInboundComposite` from the external system; confirm it invokes `C2M-IWS-CustomerSync` and the expected C2M record/outcome results.

• **Pass:** the C2M business outcome is correct and the flow is traceable in SOA/OSB tracking and IWS logging.

D2 — Outbound path reaches the external system

• **Objective:** a C2M event reaches the external system through the correct transport.

• **Steps:** trigger the C2M event; follow it through the outbound message → sender/OSB → external system;

confirm receipt.

• **Pass:** the external system received the correct payload; the outbound-message status and SOA/OSB tracking corroborate success end to end.

E. Consolidated health

E1 — Full integration report is clean

• **Objective:** one artifact proving inbound + outbound + middleware are healthy across scope.

• **Steps:** run `OSOAC2M-Report` across the in-scope interfaces; review coverage (IWS enabled/responding,

outbound configs reconciled, composites active, no staged-but-undispatched messages).

• **Pass:** report shows the whole surface healthy with no gaps for any in-scope interface — the pilot validation gate. Retain the report as change evidence.

Exit criteria for the pilot: A1–A3, B1–B4, C1–C2, D1–D2, and E1 all pass (for the mechanisms in scope), with saved script output as evidence. Only then expand to the next environment.

Troubleshooting — C2M SOA Integration

Common failures across the C2M integration surface, as **symptom** → **likely cause** → **resolution**. Start from the read-only scripts (`OSOAC2M-Discovery` + the SQL companion, `OSOAC2M-Validation`, `OSOAC2M-Report`) to localise the problem before changing anything. All fixes are non-production-first with a rollback.

Inbound Web Services (IWS)

Symptom: `iwsdeploy` **fails.**

- *Likely cause:* the WebLogic Admin Server (or target managed server) is not running, or the stored user-config/key files are missing/wrong.
- *Resolution:* start the Admin Server and target server; recreate `$SPLEBASE/etc/.wlsuserconfig` / `.wlsuserkey` with WLST `storeUserConfig`; re-run `iwsdeploy` from `$SPLEBASE/bin`.

Symptom: EAR deploys but an inbound call returns 404 / service not found.

- *Likely cause:* the specific Inbound Web Service record isn't enabled, or the EAR wasn't regenerated after enabling it.
- *Resolution:* enable the required Inbound Web Service in C2M (Requirement 4); run `initialSetup` and redeploy so the EAR reflects the service list.

Symptom: legacy XAI inbound service no longer works.

- *Likely cause:* XAI is retired — the release no longer supports it.
- *Resolution:* migrate the service to IWS per MOS **Doc ID 1644914.1**; define/enable the equivalent Inbound Web Service and redeploy.

Symptom: inbound calls fail TLS/handshake or authentication.

- *Likely cause:* the caller's certificate isn't trusted, or the endpoint's authN model doesn't match the caller.
- *Resolution:* import the certificate into the OUAF trust store with `initialSetup -i`; align the endpoint's TLS/WS-Policy/Basic/OAuth to the caller and the agreed authN standard (*verify for your version*).

Symptom: browser-UI deploy works in Test but the option is missing in Prod.

- *Cause (expected):* the development-environment flag is intentionally **off** in production.

• *Resolution:* deploy via `iwsdeploy` from `$SPLEBASE/bin` — the correct production path; leave the flag

off.

Outbound Messages

Symptom: an outbound event is raised but nothing reaches the external system.

• *Likely cause:* the Outbound Message Type's **processing method** doesn't match the intended transport,

or no Message Sender is attached.

• *Resolution:* in C2M, confirm the External System (`C2M-SOA-EXT`) + Outbound Message Type

(`C2M-OMT-SOA`) and that the processing method (SOA / real-time / JMS) matches a valid, attached Message Sender pointing at the right endpoint/queue (Requirement 5).

Symptom: messages stack up in the outbound message staging table (SOA method).

• *Likely cause:* Oracle Service Bus isn't polling — the OSB proxy (`C2M-OMT-SOA-Proxy`) is down, the

OUAF Outbound Message transport isn't installed/configured, or the external system on the message can't be reached.

• *Resolution:* confirm the OSB proxy is deployed and running, the OUAF Outbound Message transport is

installed, and the business service (`C2M-OMT-SOA-BizSvc`) target is reachable; check OSB instance tracking for faults.

Symptom: a real-time sender times out or errors.

• *Likely cause:* the external web service is unreachable/slow, or credentials/certificate are wrong.

• *Resolution:* verify connectivity and credentials from the C2M host to the external endpoint; check

the outbound-message **error status** and the sender's retry settings; fix connectivity/auth and reprocess.

Symptom: JMS outbound sender enqueues nothing / errors.

• *Likely cause:* the sender's JNDI destination/connection factory doesn't match what exists in WebLogic,

or the JMS server/store is down.

• *Resolution:* reconcile the sender's JNDI (`jms/C2MConnectionFactory` / `jms/C2MOutboundQueue`) against

the module (`OSOAC2M-Report`); confirm the integration JMS server is **RUNNING** with a healthy persistent store.

Symptom: outbound messages pile up on the JMS queue (pending count rising, zero consumers).

• *Likely cause:* no consumer (SOA/external) is attached, or the consumer errors.

• *Resolution:* confirm the consumer/OSB pipeline is running and error-free; check destination quota and

error state; drain/redeliver once the consumer is healthy. `OSOAC2M-Report` flags "backlog & 0 consumer".

SOA composites / middleware

Symptom: a composite/OSB project won't deploy.

• *Likely cause:* a missing adapter/JCA connection factory, an unresolved reference (e.g. the C2M IWS endpoint), or the wrong environment target.

• *Resolution:* confirm the JCA adapter and connection factory exist on the SOA/OSB domain, the referenced IWS endpoint is reachable, and the deploy targets the intended server; redeploy the exported SAR/JAR/configjar.

Symptom: the composite runs but the flow can't be traced.

• *Likely cause:* instance tracking/auditing is off.

• *Resolution:* enable instance tracking (composite auditing / OSB pipeline reporting) per the observability standard so every flow is visible in Enterprise Manager / the OSB console.

Symptom: a faulting instance disappears with no retry.

• *Likely cause:* no fault policy / no error destination is configured.

• *Resolution:* attach a fault policy (SOA) or route to an error destination (OSB/JMS) with bounded retries per the error-handling standard; make retries idempotent-safe (correlate on a business key).

Symptom: the composite calls C2M but authentication is rejected.

• *Likely cause:* the composite uses credentials/policy that don't match the IWS endpoint's authN.

• *Resolution:* align the composite's outbound security (Basic/WS-Security/OAuth + TLS trust) to the IWS endpoint and the agreed authN standard.

Cross-cutting

Symptom: it works in Test but fails in Prod.

• *Likely cause:* environment-specific endpoints/credentials were hard-coded, or names differ between environments.

• *Resolution:* source endpoints/credentials from the environment/secret store (never the composite/script), and reuse the **same named objects** across environments (retarget with `OSOAC2M-ScopeHelper`).

Symptom: C2M config and the deployed surface disagree.

• *Likely cause:* the C2M metadata (external system / outbound type / sender / inbound service) and the

WebLogic/SOA endpoints drifted.

• *Resolution:* run `OSOAC2M-Discovery.sql` (C2M side) and `OSOAC2M-Discovery.py` (WebLogic side) and

reconcile the names/JNDI/endpoints so both sides agree.

Escalation

If IWS deployment, outbound dispatch, or a composite still fails after the above, gather the `OSOAC2M-Discovery` snapshot (WLST + SQL), `OSOAC2M-Validation` output, the relevant server logs, and the SOA/OSB instance-tracking detail, and raise a My Oracle Support case referencing the certified version matrix (C2M ↔ WebLogic ↔ SOA/OSB ↔ DB ↔ Java) — some integration notes, adapters, and patches are MOS-gated (e.g. XAI → IWS, Doc ID 1644914.1).

Sources

• Oracle Utilities — *Polling Outbound Messages Using OSB* (processing method SOA; OUAF Outbound Message

transport; proxy/business service; staging table): https://docs.oracle.com/en/industries/utilities/meter-solution-cloud-service/264/mscs-user-guides/Topics/F1_93XAI_Polling_Outbound_Message_Using_OSB.html

• Oracle Utilities C2M — *Deploying Inbound Web Services (IWS)*:

https://docs.oracle.com/en/industries/energy-water/advanced-meter/29011/c2m-installation-guide/C2M-INSTALLATION-GUIDE/C2M_Install_Postinstall.09.09.html

• Oracle Utilities C2M — *Inbound REST Web Services* (XAI retired):

https://docs.oracle.com/en/industries/energy-water/advanced-meter/2.9.0.0.0/c2m-user-guides/C2M_29000/F1_93XAI_REST_Inbound_Web_Service_Overview.html

• Oracle® Fusion Middleware — *Understanding Technology Adapters* (JCA adapters):

https://docs.oracle.com/en/middleware/soa-suite/integration-adapters/12.2.1.3/develop-soa-adapters/overview3.html

• My Oracle Support — *Migrating from XAI to IWS*, Doc ID 1644914.1 (entitlement-gated).

Best Practices — C2M SOA Integration (supportable, low-lock-in)

Production-safe practices for the C2M integration surface. They complement the implementation steps and keep integrations **consistent, supportable, observable**, with as little lock-in as practical.

Consistency (design once, reuse everywhere)

• **One naming standard, everywhere.** Reuse the same names for External Systems, Outbound Message Types,

Message Senders, Inbound Web Services, composites, and OSB proxy/business services across every environment (`C2M-SOA-EXT`, `C2M-OMT-SOA`, `C2M-IWS-CustomerSync`, `C2MInboundComposite`, …). Consistent names make discovery/validation scripts portable and runbooks reusable.

• **Pick the mechanism by direction + timing, not by habit.** Inbound → **IWS**; synchronous outbound →

real-time HTTP/SOAP sender; near-real-time outbound → **processing method SOA** (OSB) or **JMS**. Document the choice per interface so the pattern is obvious to the next engineer.

• **Keep transformation in the middleware tier.** Do mapping/routing/enrichment in the SOA composite or

OSB pipeline, not inside C2M. C2M stays clean; the integration logic is externalised, versioned, and reusable.

Supportability

• **Migrate off XAI to IWS.** XAI is retired in the reference release; all inbound integration must be

IWS (SOAP/REST). Migrate any legacy XAI inbound service per MOS **Doc ID 1644914.1**.

• **Enable only the inbound services integrations actually need.** Every enabled IWS is surface to

secure, monitor, and support; enable by inventory (Requirement 4), not by default.

• **Deploy headless and repeatably.** Deploy `spl-web-services` with `iwsdeploy` using the stored

WebLogic user-config/key files (`$SPLEBASE/etc/.wlsuserconfig` / `.wlsuserkey`) so deploys are scriptable and identical across environments — no interactive passwords. Confirm the Admin Server is up first.

• **Version the middleware artifacts.** Keep the SOA/OSB project exports (SAR/JAR/configjar) in source

control as the deployable + rollback unit; promote the **same** export through Test and Prod rather than rebuilding per environment.

• **Turn the development-environment IWS flag OFF in production.** The "deploy from browser UI" option is

a non-production convenience; leaving it on in production is an unnecessary exposure.

Observability

• **Instance tracking on for every composite/pipeline.** So an operator can answer "did it work, and if

not, where?" from Enterprise Manager / the OSB console.

• **Use C2M outbound-message status as the outbound source of truth.** Alert on messages stuck in

error status or **staged-but-undispatched** (a sign OSB isn't polling or a sender is misconfigured).

• **Log inbound (IWS) requests** and correlate them across the composite → IWS → C2M hop so an inbound

failure can be localised.

- **Wire integrations into the environment's monitoring stack** (e.g. the APM/monitoring the site uses) for latency and error alerting per endpoint, not just aggregate health.

Security (consistent authN)

- **TLS everywhere**, with certificates in the OUAF trust store (`initialSetup -i`) and the SOA/OSB keystores.
- **One authentication model** applied to every interface — HTTP Basic over TLS, **WS-Security/WS-Policy** for SOAP, OAuth where the release supports it (*verify for your version*) — rather than per-interface improvisation.
- **Dedicated service accounts** for each integration (least privilege), and **never** put credentials or keystores in scripts, composites, or committed files — source them from the environment / secret store.

Error handling & resilience

- **Every interface has a defined failure path.** Fault policies for SOA composites, an **error destination **for JMS, bounded** retry/redelivery** limits, and monitored C2M outbound-message error status. No interface silently drops.
- **Make retries idempotent-safe.** Design inbound operations and outbound consumers so a retry cannot double-apply; correlate on a business key.
- **Bound backlogs.** Set JMS quotas and alert on rising pending counts so a stalled consumer is a bounded, visible problem — not an outage.

Low lock-in

- **Favour open, portable mechanisms.** Standard SOAP/REST and open JCA adapters (SOAP/HTTP, JMS, DB, File) over bespoke or proprietary couplings, so an interface could move to a different middleware (e.g. Oracle Integration / OIC) without re-engineering C2M.
- **Treat the middleware as replaceable.** Because transformation and routing live in named, documented composites/pipelines — not in C2M — the mediation tier can be swapped with C2M's IWS/outbound contracts unchanged.
- **Document the contract of each interface** (endpoint, payload, authN, direction, timing) so the integration is understood independently of any one product's console.

Cross-cutting

• **Non-production first, always.** Configure and validate on the pilot, then promote the *same* named objects, the *same* SOA/OSB exports, and the *same* scripts to Test and Prod.

• **Back up before disruptive changes** (domain `config.xml`, `$SPLEBASE` integration metadata, SOA/OSB project exports) and keep the restore/redeploy path as the rollback.

• **Reuse names across environments** so discovery/validation/report scripts are portable — retarget with `OSOAC2M-ScopeHelper` rather than re-authoring per environment.

• **Keep remediation dry-run by default.** Review `OSOAC2M-Remediation`'s planned actions before applying, and apply only inside a change window.

• **Retain script output as evidence** for change records and audits; a clean `OSOAC2M-Report` is the sign-off artifact.

Sources

• Oracle Utilities — *Polling Outbound Messages Using OSB* (processing method SOA; OUAF Outbound Message transport): https://docs.oracle.com/en/industries/utilities/meter-solution-cloud-service/264/mscs-user-guides/Topics/F1_93XAI_Polling_Outbound_Message_Using_OSB.html

• Oracle Utilities C2M — *Inbound REST Web Services* (IWS; XAI retired): https://docs.oracle.com/en/industries/energy-water/advanced-meter/2.9.0.0.0/c2m-user-guides/C2M_29000/F1_93XAI_REST_Inbound_Web_Service_Overview.html

• Oracle Utilities C2M — *Deploying Inbound Web Services (IWS)*: https://docs.oracle.com/en/industries/energy-water/advanced-meter/29011/c2m-installation-guide/C2M-INSTALLATION-GUIDE/C2M_Install_Postinstall.09.09.html

• Oracle® Fusion Middleware — *Understanding Technology Adapters* (JCA adapters): https://docs.oracle.com/en/middleware/soa-suite/integration-adapters/12.2.1.3/develop-soa-adapters/overview3.html

• My Oracle Support — *Migrating from XAI to IWS*, Doc ID 1644914.1 (entitlement-gated).

Deliverables Summary — SOA Integration for C2M

What this solution includes and how the pieces fit together.

Documentation

Document	Purpose
Solution Overview (`01`)	The C2M integration surface — Inbound Web Services (IWS), Outbound Messages, and SOA composites/adapters — and how they combine into one consistent, supportable, observable model.
Requirements Guide (`02`)	Every prerequisite with how to *verify* and *implement* (domain + IWS, SOA/OSB tier, integration inventory, inbound services, outbound message config, endpoints/credentials, the authN/error/observability standard).
Implementation Guide (`03`)	Exact steps: standards → deploy IWS → configure Outbound Messages → build/deploy named SOA composites/adapters → validate, pilot-first, with pass/fail gates and rollback.
Testing Guide (`05`)	Objective/steps/pass-criterion tests for inbound (IWS), outbound (real-time/SOA/JMS), composites, and end-to-end, plus a consolidated health gate.
Best Practices (`07`)	Supportable, observable, low-lock-in guidance across consistency, security, error handling, and middleware portability.
Troubleshooting Guide (`06`)	Symptom → likely cause → resolution for the common integration failures.
Executive Summary	One-page business framing for sponsors.
Global Requirements	The Oracle-category prerequisites shared by every C2M/WebLogic solution.

Automation scripts (`scripts/`)

Five scripts authored in the authentic C2M/WebLogic/SOA toolchain — **WLST (Jython), shell, and SQL** (never PowerShell). Read-only scripts are safe to run anywhere; the remediation script is **dry-run by default**.

Script	Type	Mode	What it does
`OSOAC2M-Discovery`	WLST	Read-only	Snapshots the integration surface: the `spl-web-services` (IWS) deployment and integration-relevant JMS on WebLogic, plus a companion SQL for the C2M External Systems / Outbound Message Types / Message Senders / Inbound Web Service records.

Script	Type	Mode	What it does
`OSOAC2M-Validation`	WLST	Read-only	Per-interface pass/fail checks: `spl-web-services` deployment status, integration JMS destination health, and (against expected names) IWS/outbound reconciliation.
`OSOAC2M-Report`	WLST	Read-only	Health/coverage report across scope — IWS deployed/active, integration JMS pending/consumer counts, and named-object coverage vs. the expected integration inventory.
`OSOAC2M-Remediation`	WLST	**Dry-run default**	Plans/creates the WebLogic-side integration objects that back the SOA outbound path (integration JMS server/module/CF/queue/subdeployment for `C2M-JMS-SNDR`) using the reference names — prints the plan unless an explicit apply flag is set. C2M metadata and SOA/OSB composites are configured per the Implementation Guide, not written by this script.
`OSOAC2M-ScopeHelper`	shell	Helper	Resolves and validates the target scope (interfaces, servers, connection details, expected names) and exports it so the other scripts act only in scope, portably across environments.

> *A **SQL companion** (`OSOAC2M-Discovery.sql`) is included for reading the C2M integration metadata (external systems, outbound message types + processing method, message senders, inbound web services) straight from the schema — the fastest cross-check that C2M config reconciles with the WebLogic/SOA surface.*

How they work together

`ScopeHelper` fixes the target set → `Discovery` (WLST + SQL) captures the baseline → `Validation` gates each phase → `Remediation` (dry-run first) fills the WebLogic-side gaps → `Report` produces the sign-off artifact. Reuse the same named objects and rerun the same scripts in each environment (retargeted via `ScopeHelper`) so Test and Prod get the identical, validated integration.

Outcome

The C2M integration surface designed and deployed as one coherent, consistent unit — inbound IWS, outbound messages, and SOA composites/adapters — **supportable and observable**, on **low-lock-in** patterns, proven on a pilot before scaling, with saved evidence for change control.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OSOAC2M-Discovery.py

Purpose: Connect to a running WebLogic domain and snapshot the C2M integration surface: the spl-web-services (IWS) application deployment and its targets, and any integration-relevant JMS servers / modules / destinations / connection factories that back outbound message senders. Read-only baseline evidence before any change. Set admin-url, admin-user, and admin-password for your-domain via the environment variables below. For the C2M-side metadata (external systems, outbound message types, message senders, inbound web services) use the SQL companion OSOAC2M-Discovery.sql. HOW TO RUN Set OSOAC2M_ADMIN_URL, OSOAC2M_ADMIN_USER, OSOAC2M_ADMIN_PASS (or source ScopeHelper output), then: java weblogic.WLST OSOAC2M-Discovery.py Optionally: java weblogic.WLST OSOAC2M-Discovery.py > osoac2m-discovery.out EXPECTED RESULTS Prints a sectioned snapshot (IWS DEPLOYMENT / INTEGRATION JMS) to stdout. Exit 0 on a clean read; nonzero only if it cannot connect. It never edits — safe to run any time. NOTES White-label, repeatable. Read-only: no edit()/startEdit() writes — it only reads the MBean tree, so it is safe against production. Reuse the same object names across environments (see ScopeHelper). C2M metadata is read separately via the SQL companion.

How to run: Set OSOAC2M_ADMIN_URL, OSOAC2M_ADMIN_USER, OSOAC2M_ADMIN_PASS (or source ScopeHelper output), then: java weblogic.WLST OSOAC2M-Discovery.py Optionally: java weblogic.WLST OSOAC2M-Discovery.py > osoac2m-discovery.out EXPECTED RESULTS Prints a sectioned snapshot (IWS DEPLOYMENT / INTEGRATION JMS) to stdout. Exit 0 on a clean read; nonzero only if it cannot connect. It never edits — safe to run any time. NOTES White-label, repeatable. Read-only: no edit()/startEdit() writes — it only reads the MBean tree, so it is safe against production. Reuse the same object names across environments (see ScopeHelper). C2M metadata is read separately via the SQL companion.

OSOAC2M-Discovery.sql

Purpose: Read the C2M/OUAF integration metadata straight from the application schema: the External Systems, Outbound Message Types (with processing method), Message Senders, and Inbound Web Services. This is the C2M-side of the integration surface that the WLST discovery cannot see. Read-only SELECTs only; run as the C2M read user against your-c2m-schema. Table names follow OUAF conventions; verify for your version because some names/columns differ by release (flagged inline). HOW TO RUN Connect with SQL*Plus or SQLcl as a read-capable C2M schema account, then: sqlplus your-read-user/your-password@your-c2m-db @OSOAC2M-Discovery.sql Or from SQLcl: sql /nolog -> connect ... -> @OSOAC2M-Discovery.sql EXPECTED RESULTS Four result sets printed to the spool/console: external systems, outbound message types + processing method, message senders, and inbound web services with status. No rows changed — SELECT only. If a table name differs on your release, adjust it per the OUAF data dictionary (My Oracle Support). NOTES White-label, repeatable. Strictly read-only — no INSERT/UPDATE/DELETE/DDL, safe on production. Reconcile these names against the WebLogic/SOA surface (Discovery.py) so C2M config and the deployed endpoints agree. Never store DB credentials in this file.

How to run: Connect with SQL*Plus or SQLcl as a read-capable C2M schema account, then: sqlplus your-read-user/your-password@your-c2m-db @OSOAC2M-Discovery.sql Or from SQLcl: sql /nolog ->

connect ... -> @OSOAC2M-Discovery.sql EXPECTED RESULTS Four result sets printed to the spool/console: external systems, outbound message types + processing method, message senders, and inbound web services with status. No rows changed — SELECT only. If a table name differs on your release, adjust it per the OUAF data dictionary (My Oracle Support). NOTES White-label, repeatable. Strictly read-only — no INSERT/UPDATE/DELETE/DDL, safe on production. Reconcile these names against the WebLogic/SOA surface (Discovery.py) so C2M config and the deployed endpoints agree. Never store DB credentials in this file.

OSOAC2M-Remediation.py

Purpose: Bring the WebLogic integration JMS configuration up to the expected named objects that back the C2M outbound JMS sender: create the integration JMS server, the JMS module, the connection factory, the outbound queue, and the subdeployment, using the solution reference names. By default it only PRINTS the plan; it changes nothing until you pass the explicit apply flag. IWS deployment (iwsdeploy), the C2M metadata (external systems, outbound message types, senders, inbound services), and SOA/OSB composites are done per the Implementation Guide, not written here. HOW TO RUN Dry-run (default, safe): java weblogic.WLST OSOAC2M-Remediation.py Apply for real (inside a change window, after review): OSOAC2M_APPLY=true java weblogic.WLST OSOAC2M-Remediation.py Names/targets come from environment variables (see below); source ScopeHelper first. EXPECTED RESULTS Dry-run prints "[PLAN] would create ..." lines and makes no changes (exit 0). Apply mode prints "[APPLY] created ..." inside a single edit session and activates once; objects that already exist are reported "[SKIP] exists" and left untouched. NOTES White-label, repeatable. DRY-RUN is the default — it only edits when OSOAC2M_APPLY=true, and even then only creates missing objects (idempotent), never deletes. Assign the JMS persistent store/target in your environment step; a restart is still required for JMS changes to take effect. C2M metadata + SOA/OSB deployment are done separately.

How to run: Dry-run (default, safe): java weblogic.WLST OSOAC2M-Remediation.py Apply for real (inside a change window, after review): OSOAC2M_APPLY=true java weblogic.WLST OSOAC2M-Remediation.py Names/targets come from environment variables (see below); source ScopeHelper first. EXPECTED RESULTS Dry-run prints "[PLAN] would create ..." lines and makes no changes (exit 0). Apply mode prints "[APPLY] created ..." inside a single edit session and activates once; objects that already exist are reported "[SKIP] exists" and left untouched. NOTES White-label, repeatable. DRY-RUN is the default — it only edits when OSOAC2M_APPLY=true, and even then only creates missing objects (idempotent), never deletes. Assign the JMS persistent store/target in your environment step; a restart is still required for JMS changes to take effect. C2M metadata + SOA/OSB deployment are done separately.

OSOAC2M-Report.py

Purpose: Produce a single human-readable coverage report across scope: which IWS application(s) are active and their health, each integration JMS destination with its current / pending / consumer counts (so staged-but-undispatched or backed-up outbound flows are visible), and a coverage line. Intended as the pilot sign-off artifact and change-record evidence. Reads runtime metrics; makes no changes. Point it at your-domain via the environment variables below. Pair with OSOAC2M-Discovery.sql for the C2M-metadata side. HOW TO RUN Set OSOAC2M_ADMIN_URL / OSOAC2M_ADMIN_USER / OSOAC2M_ADMIN_PASS, then: java weblogic.WLST OSOAC2M-Report.py > osoac2m-integration-report.txt EXPECTED RESULTS A sectioned report (IWS / INTEGRATION JMS METRICS) with per-object counts and an overall coverage line. Exit 0 on a clean read; nonzero only if it cannot connect. NOTES White-label, repeatable. Read-only: uses domainRuntime() metric MBeans only,

never edits — safe on production. A rising pending count with zero consumers signals an outbound flow that is staged but not being dispatched — investigate the sender / OSB proxy.

How to run: Set OSOAC2M_ADMIN_URL / OSOAC2M_ADMIN_USER / OSOAC2M_ADMIN_PASS, then: java weblogic.WLST OSOAC2M-Report.py > osoac2m-integration-report.txt EXPECTED RESULTS A sectioned report (IWS / INTEGRATION JMS METRICS) with per-object counts and an overall coverage line. Exit 0 on a clean read; nonzero only if it cannot connect. NOTES White-label, repeatable. Read-only: uses domainRuntime() metric MBeans only, never edits — safe on production. A rising pending count with zero consumers signals an outbound flow that is staged but not being dispatched — investigate the sender / OSB proxy.

OSOAC2M-ScopeHelper.sh

Purpose: Centralise the connection details, expected object names, and in-scope interface/server list so the WLST and SQL scripts (Discovery, Validation, Report, Remediation) act only where intended and use identical names across environments. Reads values from your environment or a small config file, validates them, and prints export lines you can source. Reuses the solution reference names by default; override per environment (your-env-config). HOW TO RUN Print the resolved scope: sh ./OSOAC2M-ScopeHelper.sh Load it into your shell: . ./OSOAC2M-ScopeHelper.sh --export With a config file: sh ./OSOAC2M-ScopeHelper.sh --config myenv.conf Then run e.g.: java weblogic.WLST OSOAC2M-Validation.py EXPECTED RESULTS Prints the resolved OSOAC2M_* variables and a PASS/FAIL readiness check (admin URL set, scope set, iwsdeploy present). Exit 0 when ready to run the WLST scripts; exit 1 if a required value is missing so a pipeline stops before acting out of scope. NOTES White-label, repeatable. Helper only: it neither connects nor changes anything — it just resolves scope and env vars. Set OSOAC2M_APPLY only when you intend Remediation to write; it defaults to false so the integration objects stay dry-run safe. Never commit secrets.

How to run: Print the resolved scope: sh ./OSOAC2M-ScopeHelper.sh Load it into your shell: . ./OSOAC2M-ScopeHelper.sh --export With a config file: sh ./OSOAC2M-ScopeHelper.sh --config myenv.conf Then run e.g.: java weblogic.WLST OSOAC2M-Validation.py EXPECTED RESULTS Prints the resolved OSOAC2M_* variables and a PASS/FAIL readiness check (admin URL set, scope set, iwsdeploy present). Exit 0 when ready to run the WLST scripts; exit 1 if a required value is missing so a pipeline stops before acting out of scope. NOTES White-label, repeatable. Helper only: it neither connects nor changes anything — it just resolves scope and env vars. Set OSOAC2M_APPLY only when you intend Remediation to write; it defaults to false so the integration objects stay dry-run safe. Never commit secrets.

OSOAC2M-Validation.py

Purpose: Validate the WebLogic-side integration surface against the expected named objects and print a PASS/FAIL line per check: the spl-web-services (IWS) application deployed and active, and the integration JMS server / module / connection factory / queue that back the outbound JMS sender present with the expected JNDI names. Expected names default to the solution reference names and can be overridden by environment variables. C2M-side metadata (external systems, outbound message types, senders, inbound services) is validated with the SQL companion; this script covers the deployed endpoints/JMS. HOW TO RUN Set OSOAC2M_ADMIN_URL / OSOAC2M_ADMIN_USER / OSOAC2M_ADMIN_PASS (and optional OSOAC2M_EXPECT_* / OSOAC2M_SCOPE_SERVERS from ScopeHelper), then: java weblogic.WLST OSOAC2M-Validation.py EXPECTED RESULTS One [PASS]/[FAIL] line per check and a summary count. Process exit code is 0 when every check passes, 1 when any check fails (so it gates a pipeline/change step). NOTES White-label, repeatable. Read-only: reads the config/runtime MBeans only,

no edits — safe on production. Reuse the same expected names in every environment via ScopeHelper.

How to run: Set OSOAC2M_ADMIN_URL / OSOAC2M_ADMIN_USER / OSOAC2M_ADMIN_PASS (and optional OSOAC2M_EXPECT_* / OSOAC2M_SCOPE_SERVERS from ScopeHelper), then: java weblogic.WLST OSOAC2M-Validation.py EXPECTED RESULTS One [PASS]/[FAIL] line per check and a summary count. Process exit code is 0 when every check passes, 1 when any check fails (so it gates a pipeline/change step). NOTES White-label, repeatable. Read-only: reads the config/runtime MBeans only, no edits — safe on production. Reuse the same expected names in every environment via ScopeHelper.

no edits — safe on production. Reuse the same expected names in every environment via ScopeHelper.